ΠΡΟΓΡΑΜΜΑΤΙΣΤΙΚΗ ΑΣΚΗΣΗ 1-Α

OpenGL – Δημιουργία και διαχείριση

δισδιάστατων μοντέλων

Γραφικά Υπολογιστών και Συστήματα Αλληλεπίδρασης

Ακαδημαϊκό Έτος: 2026- 2027

Στοιχεία ομάδας:

Χριστόδουλος Αψιώτης ΑΜ: 5398

Χαράλαμπος Κουνναπής ΑΜ: 5401

Ημερομηνία: Παρασκευή, 23 Οκτωβρίου 2026

Περιεχόμενα

1. Περιγραφή της εργασίας 4

1.1 Ερώτημα (i) – Δημιουργία Βασικού Παραθύρου 900 x 900 4

1.1.1 Περιγραφή και Τρόπος Υλοποίησης 4

1.1.2 Δημιουργία παραθύρου και καθορισμός διαστάσεων 4

1.2 Ερώτημα (i) - Καθορισμός του μπλε χρώματος στο φόντο 4

1.2.1 Περιγραφή και Τρόπος Υλοποίησης 4

1.3 Ερώτημα (i) - Τερματισμός της εφαρμογής με το πλήκτρο Q 5

1.3.1 Περιγραφή και Τρόπος Υλοποίησης 5

1.4 Ερώτημα (i) – Λειτουργία Αλγορίθμου 6

1.4.1 Περιγραφή 6

1.5 Ερώτημα (ii) – Δημιουργία και Σχεδίαση των τεσσάρων μοντέλων 6

1.5.1 Περιγραφή και τρόπος υλοποίησης 6

1.5.2 Δημιουργία τετράγωνου – Συνάρτηση addSquare() 7

1.5.3 Δημιουργία των μοντέλων – Συνάρτηση createModel() 8

1.5.4 Προσδιορισμός συντεταγμένων 10

1.5.5 Τυχαία επιλογή μοντέλου 11

1.6 Ερώτημα (iii) – Τυχαία τοποθέτηση μέχρι τρία μοντέλα χωρίς επικάλυψη 12

1.6.1 Περιγραφή και τρόπος υλοποίησης 12

1.6.2 Αποθήκευση των μοντέλων 12

1.6.3 Έλεγχος του πλήκτρου S και του ορίου τριών μοντέλων 13

1.6.4 Υπολογισμός Τυχαίων Συντεταγμένων 14

1.6.5 Έλεγχος επικάλυψης - Συνάρτηση overlap() 14

1.6.6 Εύρεση νέας θέσης με do/while 15

1.6.7 Συγκέντρωση των κορυφών και σχεδίαση μοντέλων 16

1.6.8 Λειτουργία αλγορίθμου 17

1.7 Ερώτημα (iv) – διαγραφή παλιότερο μοντέλου με το κουμπί D 18

1.7.1 Περιγραφή και τρόπος υλοποίησης 18

1.7.2 Έλεγχος του κουμπιού D 18

1.7.3 Διαγραφή του μοντέλου 19

1.7.4 Λειτουργία αλγορίθμου 20

2. Πληροφορίες σχετικά με την υλοποίηση 20
2.1 Λειτουργικό σύστημα 20
2.2 Περιβάλλον ανάπτυξης 20
2.3 Ιδιαιτερότητες στον τρόπο εκτελέσεις 21
2.4 Άλλες ιδιαιτερότητες 21
3. Πληροφορίες σχετικά με την υλοποίηση 21
4. Αναφορές 22
4.1 Επίλυση προβλήματος τίτλου 22

1. Περιγραφή της εργασίας

1.1 Ερώτημα (i) – Δημιουργία Βασικού Παραθύρου 900 x 900

1.1.1 Περιγραφή και Τρόπος Υλοποίησης

Στο πρώτο ερώτημα μας ζητήθηκε να δημιουργήσουμε ένα βασικό παράθυρο στην OpenGL με διαστάσεις 900x900, με φόντο μπλε σκούρο χρώμα. Επιπλέων το παράθυρο εμφανίζει τον τίτλο στα ελληνικά και τους αριθμούς μητρώου των δυο μέλλω της ομάδας και τερματίζει με το πλήκτρο Q. Για την υλοποίηση χρησιμοποιήθηκε ο αρχικός κώδικας που μας δόθηκε για την άσκηση με τις απαραίτητες τροποποιήσεις στις παραμέτρους του παραθύρου και στο χρώμα του φόντου.

1.1.2 Δημιουργία παραθύρου και καθορισμός διαστάσεων

Αρχείο:	Source-1A.cpp
Γραμμές Κώδικα:	204 – 206
window = glfwCreateWindow(900, 900, «Εργασία 1A – 5398 – 5401», NULL, NULL);	

```
204     // Open a window and create its OpenGL context
205     // Allazoume to megethos tou parathirou apo 750x750 se 900x900
206     window = glfwCreateWindow(900, 900, "Εργασια 1A - 5398 - 5401", NULL, NULL);
```

Για την δημιουργία του παραθύρου χρησιμοποιήθηκε η συνάρτηση glfwCreateWindow() της βιβλιοθήκης GLFW. Οι δυο πρώτες παράμετροι αλλάζουν το πλάτος και το ύψος του παραθύρου σε 900x900 pixels αντίστοιχα. Η τρίτη παράμετρος αλλάζει τον τίτλο ενώ οι δυο τελευτές παράμετροι είναι κενές καθώς δεν χρησιμοποιείται η λειτουργία πλήρους οθόνης.

1.2 Ερώτημα (i) - Καθορισμός του μπλε χρώματος στο φόντο

1.2.1 Περιγραφή και Τρόπος Υλοποίησης

Αρχείο:	Source-1A.cpp
Γραμμές Κώδικα:	229 - 231
glClearColor(0.05f, 0.05f, 0.45f, 1.0f);	

```
229     // blue background
230     // Vazoume skouro mple chroma sto kouti
231     glClearColor(0.05f, 0.05f, 0.45f, 1.0f);
```

Για τον καθορισμό του φόντου χρησιμοποιήθηκε η συνάρτηση glClearColor().

Η συνάρτηση δέχεται τέσσερεις παράμετρούς οι οποίες αντιστοιχούν στις συνιστώσες Red, Green, Blue και Alpha με τιμές από 0 έως 1.

Οι τιμές που έχουμε προσθέσει εμείς για τις πρώτες τρεις παράμετροι επιλέχθηκαν έτσι ώστε να προκύπτει ένα σκούρο μπλε φόντο, ενώ η τιμή που έχουμε βάλει στην τέταρτη παράμετρο είναι η μέγιστη τιμή που μπορεί να πάρει.

Επιπλέον στην γραμμή 322 χρησιμοποιήσαμε την εντολή glClear(GL_COLOR_BUFFER_BIT) ώστε να καθορίζεται η περιοχή σχεδιάσεις με το καθορισμένο χρώμα.

```
321     // Clear the screen
322     glClear(GL_COLOR_BUFFER_BIT);
```

1.3 Ερώτημα (i) - Τερματισμός της εφαρμογής με το πλήκτρο Q

1.3.1 Περιγραφή και Τρόπος Υλοποίησης

Αρχείο:	Source-1A.cpp
Γραμμή Κώδικα:	357
While(glfwGetKey(windows, GLFW_KEY_Q) != GLFW_PRESS && glfwwindowShouldClose(window) == 0);	

```
357 } while (glfwGetKey(window, GLFW_KEY_Q) != GLFW_PRESS && glfwWindowShouldClose(window) == 0);
```

Για τον τερματισμό της εφαρμογής χρησιμοποιήσαμε τη συνάρτηση glfwGetKey() η οποία ελέγχει την κατάσταση του κουμπιού Q.

Η παράμετρος window αναφέρεται στο παράθυρο της εφαρμογής, η παράμετρος GLFW_KEY_Q προσδιορίζει το κουμπί Q και η σταθερά GLFW_PRESS δηλώνει ότι το κουμπί είναι πατημένο. Η συνθήκη glfwGetKey(window, GLFW_KEY_Q) != GLFW_PRESS ελέγχει αν το κουμπί Q δεν είναι πατημένο. Επιπλέον η συνάρτηση glfwWindowShouldClose(window) ελέγχει αν ο χρήστης έχει ζητήσει να κλείσει το παράθυρο. Όσο και οι δύο συνθήκες ισχύουν η επανάληψη do/while συνεχίζεται μέχρι ο χρήστης πατήσει το κουμπί Q με αποτέλεσμα να κλείσει το παράθυρο.

1.4 Ερώτημα (i) – Λειτουργία Αλγορίθμου

1.4.1 Περιγραφή

Αρχικά το πρόγραμμα πραγματοποιεί την απαραίτητη αρχικοποίηση της βιβλιοθήκης GLFW με την συνάρτηση glfwInit() και της βιβλιοθήκης GLEW με την συνάρτηση glewInit(). Στην συνέχεια δημιουργείτε το παράθυρο διαστάσεων 900x900 με την συνάρτηση glfwCreateWindow() και καθορίζεται το σκούρο μπλε χρώμα φόντου με την συνάρτηση glClearColor().

Αφού ολοκληρωθούν οι απαραίτητες αρχικοποιήσεις το πρόγραμμα μπαίνει στον βρόγχο do/while μέσα στον οποίο πραγματοποιείτε η ανανέωση του παραθύρου και ο έλεγχος των πλήκτρων.

Σε κάθε επανάληψη χρησιμοποιείτε η εντολή glClear(GL_COLOR_BUFFER_BIT) η οποία καθαρίζει την περιοχή σχεδίασης με το χρώμα που έχουμε καθορίσει.

Η διαδικασία επαναλαμβάνεται μέχρι να πατηθεί το πλήκτρο Q από τον χρήστη ή ο χρήστης να κλήση το παράθυρο.Τέλος το πρόγραμμα καλεί την συνάρτηση glfwTerminate() ώστε να τερματίσει την λειτουργία της GLFW.

1.5 Ερώτημα (ii) – Δημιουργία και Σχεδίαση των τεσσάρων μοντέλων

1.5.1 Περιγραφή και τρόπος υλοποίησης

Στο δεύτερο ερώτημα της εργασίας μας ζητήθηκε να δημιουργήσουμε τέσσερα διαφορετικά μοντέλα A, B, C και D, τα οποία σχηματίζονται από τετράγωνα πλευράς n=1. Σύμφωνα με την εκφώνηση κάθε τετράγωνο αποτελείτε από δυο τρίγωνα.

Το πρόγραμμα αρχικά εμφανίζει ένα άδειο παράθυρο με σκούρο μπλε φόντο και με το πάτημα του κουμπιού S επιλέγεται τυχαία ένα από τα τέσσερα μοντέλα και δημιουργείτε στο παράθυρο.

Για αυτή την υλοποίηση, δημιουργήθηκαν δυο συναρτήσεις, η addSquare() η οποία είναι υπεύθυνη για τον υπολογισμό και την αποθήκευση των κορυφών ενός τετραγώνου και η createModel() η οποία χρησιμοποιεί την addSquare() για να σχηματίσει τα τέσσερα διαφορετικά μοντέλα.

Η σχεδίαση των τετραγώνων γίνεται με τρίγωνα ενώ οι συντεταγμένες των κορυφών υπολογίζονται με βάση τη θέση αναφοράς και το μήκος της πλευράς.

1.5.2 Δημιουργία τετράγωνου – Συνάρτηση addSquare()

Αρχείο:	Source1A.cpp
Γραμμές Κώδικα:	134 – 145

```
void addSquare(vector<GLfloat>& vertices, float x, float y, float n) {
    //proto trigono
    vertices.insert(vertices.end(), {
        x, y, 0.f,
        x, y - n, 0.0f,
        x + n, y, 0.0f });
    //deftero rtigono
    vertices.insert(vertices.end(), {
        x, y - n, 0.0f,
        x + n,y - n, 0.0f,
        x + n, y, 0.0f });
}
```

Η συνάρτηση addSquare() δέχεται ως παράμετρο ένα πίνακα κορυφών (vertices), τις συντεταγμένες x και y της πάνω αριστερής κορυφής του και το μήκος της πλευράς n.

Για την δημιουργία του τετραγώνου χρησιμοποιούνται δυο τρίγωνα, κάθε ένα από τα οποία αποτελούνται από τρεις κορυφές.

Η πρώτη τριάδα υπολογίζεται από τις συντεταγμένες :

1. (x, y, 0)

2. (x, y-n, 0)

3. (x+n, y, 0)

Η δεύτερη τριάδα κορυφών υπολογίζεται από τις συντεταγμένες:

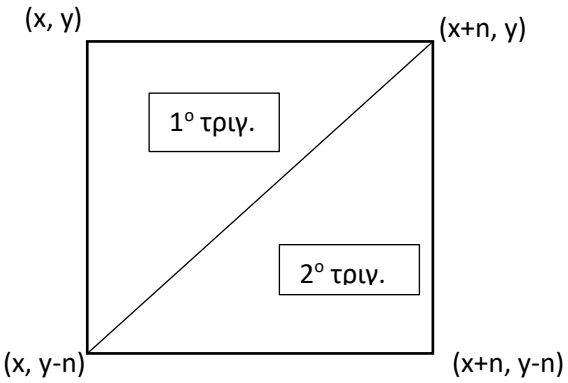
1. (x, y-n, 0)

2. (x+n, y-n, 0)

3. (x+n, y, 0)

Με αυτό τον τρόπο δημιουργείτε ένα τετράγωνο πλευράς n, χρησιμοποιώντας συνολικά έξι κορυφές αλλά λόγο του ότι βρισκόμαστε σε 2D διαστάσεις η συντεταγμένη z δεν χρησιμοποιείτε οπόταν την βάζουμε μηδέν.

Οι συντεταγμένες αποθηκεύονται στον πίνακα vertices με την εντολή vertices.insert(), έτσι ώστε να μπορούν να χρησιμοποιηθούν στην συνέχεια από την opengl για την σχεδίαση.



1.5.3 Δημιουργία των μοντέλων – Συνάρτηση createModel()

Αρχείο:	Source-1A.cpp
Γραμμές Κώδικα:	147 – 172

```
147  //dimiourgia ton tessaron montelon
148  vector<GLfloat> createModel(int type, float x, float y) {
149      vector<GLfloat> vertices;
150      float n = 1.0f;
151      //1o montelo
152      addSquare(vertices, x, y, n);
153
154      //2o montelo
155      if (type == 1) {
156          addSquare(vertices, x + n, y, n);
157      }
158
159      //3o montelo
160      else if (type == 2) {
161          addSquare(vertices, x, y - n, n);
162          addSquare(vertices, x, y - 2 * n, n);
163
164      }
165
166      //4o montelo
167      else if (type == 3) {
168          addSquare(vertices, x + n, y, n);
169          addSquare(vertices, x, y - n, n);
170      }
171      return vertices;
172  }
```

Η συνάρτηση createModel() δέχεται ως παράμετρο τον τύπο του μοντέλου (type) και τις συντεταγμένες x και y.

Αρχικά δημιουργεί ένα πίνακα vertices, στον οποίο αποθηκεύονται οι κορυφές όλων των τετραγώνων που σχηματίζουν το μοντέλο και το μήκος της πλευράς ορίζεται με την εντολή «float n = 1.0f;»

Η δημιουργία τον μοντέλων γίνεται ως εξής:

Μοντέλο Α – τετράγωνο

Το μοντέλο Α αποτελείτε από ένα τετράγωνο. Η πάνω αριστερή κορυφή του βρίσκεται στην θέση (x,y).

Χρησιμοποιείτε η εντολή « addSquare(vertices, x, y, n); » η οποία βρίσκεται στην γραμμή 152.

Μοντέλο Β – δυο οριζόντια τετράγωνα

Το μοντέλο Β αποτελείτε από δυο τετράγωνα που βρίσκονται οριζόντια.

Το πρώτο τετράγωνο έχει πάνω αριστερή κορυφή (x,y) ενώ το δεύτερο τοποθετείτε κατά μια μονάδα δεξιά του στην θέση (x+n, y).

Η δημιουργία του γίνεται στις γραμμές 155 – 157.

Μοντέλο C – τρία κατακόρυφα τετράγωνα

Το μοντέλο C αποτελείτε από τρία κάθετα τετράγωνα.

Οι πάνω αριστερές κορυφές τους βρίσκονται στις θέσεις:

- Πρώτο τετράγωνο: (x, y)
- Δεύτερο τετράγωνο: (x, y-n)
- Τρίτο τετράγωνο: (x, y-2n)

Η δημιουργία του δευτέρου και τρίτου τετραγώνου γίνετε στις γραμμές 160 – 162

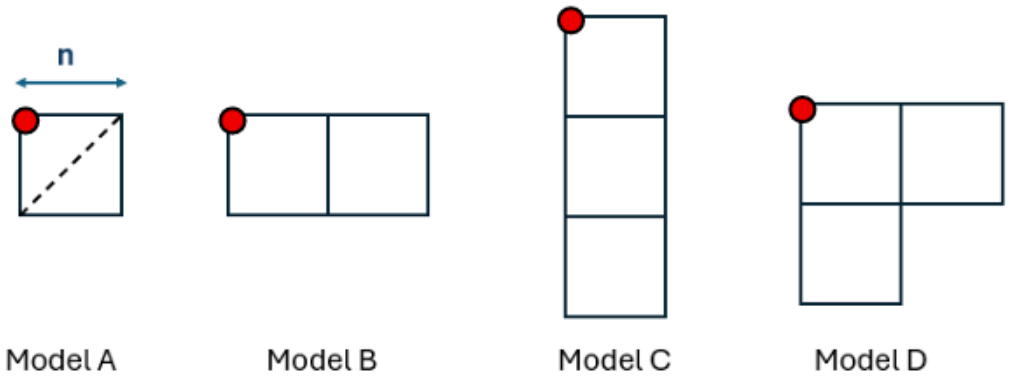
Μοντέλο D – τρία τετράγωνα σε σχήμα Γ

Το τέταρτο και τελευταίο μοντέλο αποτελείτε από τρία τετράγωνα που σχηματίζουν το γράμμα Γ.

Οι πάνω αριστερές κορυφές τους βρίσκονται στις θέσεις:

- Πρώτο τετράγωνο: (x,y)
- Δεύτερο τετράγωνο: (x+n, y)
- Τρίτο τετράγωνο: (x, y-n)

Η δημιουργία του δευτέρου και τρίτου τετραγώνου γίνετε στις γραμμές 167 – 169.



1.5.4 Προσδιορισμός συντεταγμένων

Αρχείο:	Source-1A.cpp
Γραμμές Κώδικα:	134 – 172

```
134  void addSquare(vector<GLfloat>& vertices, float x, float y, float n) {
135      //proto trigono
136      vertices.insert(vertices.end(), {
137          x, y, 0.f,
138          x, y - n, 0.0f,
139          x + n, y, 0.0f });
140      //deftero rtigono
141      vertices.insert(vertices.end(), {
142          x, y - n, 0.0f,
143          x + n,y - n, 0.0f,
144          x + n, y, 0.0f });
145  }
146
147  //dimiourgia ton tessaron montelon
148  vector<GLfloat> createModel(int type, float x, float y) {
149      vector<GLfloat> vertices;
150      float n = 1.0f;
151      //1o montelo
152      addSquare(vertices, x, y, n);
153
154      //2o montelo
155      if (type == 1) {
156          addSquare(vertices, x + n, y, n);
157      }
158
159      //3o montelo
160      else if (type == 2) {
161          addSquare(vertices, x, y - n, n);
162          addSquare(vertices, x, y - 2 * n, n);
163
164      }
165
166      //4o montelo
167      else if (type == 3) {
168          addSquare(vertices, x + n, y, n);
169          addSquare(vertices, x, y - n, n);
170      }
171      return vertices;
172  }
```

Ως σημείο αναφοράς χρησιμοποιείτε η πάνω αριστερή κορυφή του πρώτου τετραγώνου σε κάθε μοντέλο.

Επόμενος οι συντεταγμένες των υπόλοιπόν τετραγώνων προσδιορίζονται με βάση το μήκος της πλευράς n=1 και τη σχετική τους θέση ως προς το πρώτο τετράγωνο.

- Μοντέλο A: (x, y)
- Μοντέλο B: (x,y), (x+n, y)
- Μοντέλο C: (x,y), (x, y-n), (x, y-2n)
- Μοντέλο D: (x,y), (x+n,y), (x,y-n)

Στην συνέχεια για κάθε τετράγωνο, οι έξι κορυφές των δυο τριγώνων υπολογίζονται μέσω της συνάρτησης addSquare().

Με αυτό τον τρόπο, οι συντεταγμένες των μοντέλων δεν καταχωρούνται ξεχωριστά αλλά υπολογίζονται παραμετρικά.

1.5.5 Τυχαία επιλογή μοντέλου

Αρχείο:	Source-1A.cpp
Γραμμές Κώδικα:	276 – 277

```
276     //tichaioi arithmoi
277     srand(static_cast<unsigned int>(time(0)));
```

Σε αυτό το σημείο θέλουμε όταν πατάμε το κουμπί S το πρόγραμμα να επιλεγεί τυχαία ένα από τα τέσσερα μοντέλα και να τα σχεδιάζει στο παράθυρο.

Για την τυχαία επιλογή χρησιμοποιήσαμε την εντολή «srand() % 4» η οποία κάνει generate ένα ακέραιο αριθμό από το 0 έως το 3 οπού κάθε αριθμός αντιστοιχεί σε ένα διαφορετικό μοντέλο.

Ανάλογα με τον αριθμό που επιλέγεται καλείται η συνάρτηση createModel(), η οποία δημιουργεί το μοντέλο χρησιμοποιώντας την addSquare().

1.6 Ερώτημα (iii) – Τυχαία τοποθέτηση μέχρι τρία μοντέλα χωρίς επικάλυψη

1.6.1 Περιγραφή και τρόπος υλοποίησης

Στο τρίτο μέρος της εργασίας τροποποιήσαμε των κώδικα μας έτσι ώστε όταν πατάμε το κουμπί S να δημιουργείτε ένα νέο μοντέλο σε τυχαίο σημείο μέσα στο παράθυρο. Επίσης πρέπει να επιτρέπει την σχεδίαση μέχρι τριών μοντέλων χωρίς να υπάρχει επικάλυψη μεταξύ τους.

Για την υλοποίηση χρησιμοποιήσαμε τον πίνακα models, στον οποίο αποθηκεύουμε τις κορυφές κάθε μοντέλου που δημιουργείτε έτσι ώστε να μπορούμε να διατηρήσουμε τα προηγούμενα μοντέλα στο παράθυρο όταν δημιουργούμε ένα νέο.

Ακόμη για τον έλεγχο της επικάλυψης δημιουργήσαμε την συνάρτηση overlap() η οποία ελέγχει αν ένα νέο μοντέλο επικαλύπτεται με κάποιο από αυτά που είδη υπάρχουν.

Εάν βρεθεί επικάλυψη, χρησιμοποιούμε την επανάληψη do/while, ώστε να υπολογίζεται νέα τυχαία θέση μέχρι να βρεθεί μια στην οποία το μοντέλο δεν επικαλύπτεται.

Τέλος ελέγχουμε ότι ο αριθμός των μοντέλων δεν είναι περισσότερα από τρία με την συνθήκη models.size()<3

1.6.2 Αποθήκευση των μοντέλων

Αρχείο:	Source-1A.cpp
Γραμμές Κώδικα:	276 – 277

```
266     // oloes oi korifes
267     vector<GLfloat> shape;
268
269     //apothikevi mechri 3 montela
270     vector<vector<GLfloat>> models;
271
272     bool previousS = false;
273     bool previousD = false;
274
```

Για να αποθηκεύουμε τα μοντέλα που δημιουργούνται χρησιμοποιήσαμε ένα δισδιάστατο πίνακα με όνομα models, vector<vector<GLfloat>> models;

Ο συγκεκριμένος πίνακας επιτρέπει την αποθήκευση των κορυφών κάθε μοντέλου ξεχωριστά. Με αυτό τον τρόπο μπορούμε να προσθέσουμε στον πίνακα ένα νέο μοντέλο που δημιουργείτε χωρίς να χρειαστεί η αντικατάσταση των προηγουμένων μοντέλων.

Επιπλέων χρησιμοποιήσαμε την εντολή vector<GLfloat> shape; , οπού την χρησιμοποιούμε για να συγκεντρώσουμε όλες τις κορυφές τον μοντέλων που έχουν αποθηκευτεί ώστε να μπορούν να σχεδιαστούν στο παράθυρο.

Η βασική διαφορά που έχουν οι δυο πίνακες είναι ότι ο models αποθηκεύει κάθε μοντέλο ξεχωριστά ενώ ο shape αποθηκεύει όλες τις κορυφές που χρειάζονται.

1.6.3 Έλεγχος του πλήκτρου S και του ορίου τριών μοντέλων

Αρχείο:	Source-1A.cpp
Γραμμές Κώδικα:	272,283-288 και 305-306

```
272     bool previousS = false;
```

```
304         //neo montelo otan patiete to s kathe fora
305         previousS = currentS;
306
```

```
283     //elechos gia to pliktro S
284     bool currentS = (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS);
285
286     //dimiourgi mechri 3 montela
287     if (currentS && !previousS && models.size() < 3) {
```

Για να ελέγχουμε ποτέ ο χρήστης πατάει το πλήκτρο S, χρησιμοποιήσαμε την συνάρτηση glfwGetKey().

Στη γραμμή 285 χρησιμοποιούμε την εντολή,bool currentS = (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS);

Η currentS αποθηκεύει την τρέχουσα κατάσταση του κουμπιού S. Εάν το κουμπί είναι πατημένο η μεταβλητή παίρνει τιμή true αλλιώς παίρνει την τιμή false.

Στην γραμμή 288 έχουμε τον έλεγχο if (currentS && !previousS && models.size() < 3), ο οποίος παίρνει τρεις συνθήκες και τις ελέγχει.

Αρχικά ελέγχει αν το κουμπί είναι πατημένο με την currentS, στην συνέχεια ελέγχει αν το κουμπί δεν ήταν πατημένο στην προηγούμενη επανάληψη με το !previousS και τέλος ελέγχει αν ο αριθμός των αποθηκευμένων μοντέλων είναι κάτω από 3 με το models.size()<3.

Ακόμη στην γραμμή 272 αρχικοποιείται η τιμή previousS με false και στην γραμμή 305 την καλούμε. Έτσι ελέγχουμε αν δημιουργείτε ένα μοντέλο ανά πάτημα και όχι συνεχόμενα όσο το κουμπί είναι πατημένο.

1.6.4 Υπολογισμός Τυχαίων Συντεταγμένων

Αρχείο:	Source-1A.cpp
Γραμμές Κώδικα:	291 – 299

```
291             //to do ektelei dimiourgia montelou se tichea thesi
292             do {
293                 //tchees sintetagmenes
294                 float x = rand() % 19 - 9;
295                 float y = rand() % 20 - 8;
296
297                 //dimioiurgia tou montelou
298                 newModel = createModel(modelType, x, y);
299             } while (overlap(newModel, models));//eno to while elechi an eiparchi epikalipsi
```

Για να τοποθετηθούν τυχαία τα μοντέλα πάνω στο παράθυρο χρησιμοποιήσαμε την συνάρτηση rand() η οποία παράγει τυχαίες τιμές για τις συντεταγμένες x και y.

Επίσης χρησιμοποιήσαμε τις εντολές float x = rand() % 19-9; και float y = rand() % 20-8; Το πρώτο μέρος (rand()) υπολογίζει την τυχαία συντεταγμένη ενώ το δεύτερο μέρος και των δυο εντολών είναι η πράξη που μας βοηθά να μένουμε στο διάστημα [-9,9] και [-8, 11] αντίστοιχα.

Ακόμη με την συνάρτηση newmodel = createModel(modelType, x, y); δημιουργείτε το μοντέλο που έχει επιλεγεί τυχαία.

1.6.5 Έλεγχος επικάλυψης - Συνάρτηση overlap()

Αρχείο:	Source-1A.cpp
Γραμμές Κώδικα:	291 – 299

```
174 //elechos gia epikalipsi 2 montelon
175 bool overlap(const vector<GLfloat>& newModel, const vector<vector<GLfloat>>& models) {
176     for (const auto& model : models) {
177         for (int i = 0; i < newModel.size(); i += 18) {
178             for (int j = 0; j < model.size(); j += 18) {
179                 if (newModel[i] == model[j] && newModel[i + 1] == model[j + 1]) {
180                     return true;
181                 }
182             }
183         }
184     }
185     return false;
186 }
```

Για να αποφύγουμε την επικάλυψη μεταξύ των μοντέλων δημιουργήσαμε τη συνάρτηση overlap(), η οποία ελέγχει αν ένα νέο μοντέλο βρίσκετε στην ιδιά θέση με ένα προηγούμενο που ήδη υπάρχει.

Η συνάρτηση παίρνει δύο παραμέτρους την newmodel η οποία περιέχει τις κορυφές του νέου μοντέλου και την models η οποία περιέχει τις κορυφές τον μοντέλων που υπάρχουν ήδη

Αρχικά στην πρώτη επανάληψη εξετάζουμε ένα προς ένα τα μοντέλα που ήδη υπάρχουν στον πίνακα models και στην συνέχεια χρησιμοποιούμε τις επόμενες δυο επαναλήψεις για να συγκρίνουμε τα τετράγωνα του νέου μοντέλου με τα τετράγωνα των μοντέλων που ήδη υπάρχουν.

Λόγο του ότι κάθε τετράγωνο αποτελείτε από δυο τρίγωνα και κάθε κορυφή έχει τρεις συντεταμένες βάλαμε ότι τα i και j των δυο επαλείψεων ότι ισούται με 18.

Στην συνέχεια υπάρχει μια συνθήκη if η οποία συγκρίνει τις συντεταγμένες x και y της πάνω αριστερής κορυφής το νέου μοντέλου με την κορυφή του υπάρχοντα μοντέλου.

Εάν οι δυο συντεταγμένες είναι ιδίες τότε αυτό σημαίνει ότι τα δυο τετράγωνα βρίσκονται στην ιδιά θέση με αποτέλεσμα να υπάρχει επικάλυψη και η συνάρτηση επιστρέφει true.

Αν όμως ολοκληρωθούν όλες οι συγκρίσεις χωρίς να βρεθεί επικάλυψη τότε εκτελείτε η εντολή return false; Με αποτέλεσμα το νέο μοντέλο να μην επικαλύπτετε με κάποιο από τα υπάρχοντα και να τοποθετηθεί στην θέση.

1.6.6 Εύρεση νέας θέσης με do/while

Αρχείο:	Source-1A.cpp
Γραμμές Κώδικα:	286 – 302

```
286         //dimiourgi mechri 3 montela
287         if (currentS && !previousS && models.size() < 3) {
288             //tichea epilogi montelou
289             int modelType = rand() % 4;
290             vector<GLfloat> newModel;
291             //to do ektelei dimiourgia montelou se tichea thesi
292             do {
293                 //tchees sintetagmenes
294                 float x = rand() % 19 - 9;
295                 float y = rand() % 20 - 8;
296 
297                 //dimioiurgia tou montelou
298                 newModel = createModel(modelType, x, y);
299             } while (overlap(newModel, models));//eno to while elechi an eiparchi epikalipsi
300             //apothikevei to montelo
301             models.push_back(newModel);
302         }
```

Σε αυτό το σημείο για να εξασφαλίσουμε ότι τα μοντέλα δεν επικαλύπτονται μεταξύ τους χρησιμοποιήσαμε μια επανάληψη do/while η οποία προσθετή το νέο μοντέλο σε τυχαία θέση και στην συνέχεια γίνετε έλεγχος επικάλυψης μεταξύ των μοντέλων που υπάρχουν ήδη.

Στην γραμμή 290 γίνετε δήλωση ενός προσωρινού πίνακα ο οποίος χρησιμοποιείτε για την αποθήκευση των κορυφών του νέου μοντέλου πριν αυτό εισαχθεί στον πίνακα models.

Στην συνέχεια μέσα στην επανάληψη do/while υπολογίζονται οι τυχαίες συντεταγμένες x και y οι οποίες καθορίζουν την αρχή κάθε νέου μοντέλου.

Μετά καλείτε η συνάρτηση createModel() η οποία προσθέτει το νέο μοντέλο στην συγκεκριμένη θέση και αποθηκεύει της συντεταγμένες του στο πίνακα newmodel.

Αφού έχει προστεθεί το μοντέλο εκτελείτε η συνθήκη while (overlap(newModel, models)); η οποία ελέγχει αν υπάρχει επικάλυψη. Αν στην περίπτωση που υπάρχει επικάλυψη ξανά επαναλαμβάνετε ο βρόχος μέχρι να βρεθεί θέση για την δημιουργία του μοντέλου που δεν υπάρχει επικάλυψη.

Τέλος χρησιμοποιήσαμε την εντολή models.push_back(newModel); η οποία αποθηκεύει το μοντέλο στον πίνακα models αφού έχει ολοκληρωθεί πρώτα ο έλεγχος.

1.6.7 Συγκέντρωση των κορυφών και σχεδίαση μοντέλων

Αρχείο:	Source-1A.cpp
Γραμμές Κώδικα:	315-321, 332 – 341 και 343 – 349

```
315
316         //to parathiro xekina adio
317         shape.clear();
318
319         for (const auto& model : models) {
320             shape.insert(shape.end(), model.begin(), model.end());
321         }
```

```
343         // Draw the triangle !
344         // Valame tin pio kato grammi kodika se scholia giati sto ii) thelei na anigi ena adio mple
345         // parathiro
346         //glDrawArrays(GL_TRIANGLES, 0, 3 * 2); // 3 vertices for each triangle -> 6 vertices total for 2 triangles
347         //tin antikathistoume me tin pio kato etsi oste na ginete schediasi olon ton
348         //korifon tou montelou
349         glDrawArrays(GL_TRIANGLES, 0, static_cast<GLsizei>(shape.size() / 3));
```

```
332     //metafora korifon
333     glBufferData(GL_ARRAY_BUFFER, shape.size() * sizeof(GLfloat), shape.empty() ? nullptr : shape.data(), GL_DYNAMIC_DRAW);
334     glVertexAttribPointer(
335         0,                  // attribute 0, must match the layout in the shader.
336         3,                  // size
337         GL_FLOAT,           // type
338         GL_FALSE,           // normalized?
339         0,                  // stride
340         (void*)0            // array buffer offset
341     );
```

Για να δημιουργούνται όλα τα μοντέλα που έχουν επιλεγεί χρησιμοποιούμε τον πίνακα shape στον οποίο συγκεντρώνουμε τις κορυφές των μοντέλων που είναι αποθηκευμένες στον πίνακα models.

Αρχικά χρησιμοποιούμε την εντολή shape.clear() όπου αδειάζει τον πίνακα από τις κορυφές που περιείχε στην προηγουμένη επανάληψη.

Στην συνέχεια χρησιμοποιήσαμε ένα for loop ο οποίος περνάει από όλα τα μοντέλα που βρίσκονται στον πίνακα models. Για κάθε μοντέλο η εντολή shape.inster() προσθέτει τις κορυφές του στο τέλος του πίνακα shape.

Μετά γίνετε μεταφορά των κορυφών στην OpenGL με την χρήση της συνάρτησης glBufferData() η οποία χρησιμοποιείται για την μεταφορά τον κορυφών από τον πίνακα shape στον buffer τις OpenGL

Η παράμετρος shape.size() * sizeof(GLfloat) υπολογίζει το μέγεθος, shape.empty() ? nullptr : shape.data() ελέγχει αν ο πίνακας είναι άδειος και αν είναι άδειος χρησιμοποιεί την τιμή nullptr και η παράμετρος GL_DYNAMIC_DRAW δηλώνει ότι τα δεδομένα του buffer μπορεί να αλλάξουν κατά την λειτουργεία του προγράμματος.

Επίσης η συνάρτηση glVertexAttribPointer() καθορίζουν ότι κάθε κορυφή περιχέει τρεις συντεταγμένες τύπου float.

Στην συνέχεια γίνεται η σχεδίαση των μοντέλο με την εντολή glDrawArrays(GL_TRIANGLES, 0, static_cast<GLsizei>(shape.size() / 3));

Η οποία χρησιμοποιεί τρεις παραμέτρους, την GL_TRIANGLES που δηλώνει ότι η σχεδίαση γίνεται με τρίγωνα, το μηδέν ότι ξεκινά από την πρώτη κορυφή και την shape.size() / 3 που υπολογίζει τον συνολικό αριθμό κορυφών που πρέπει να σχεδιαστούν.

1.6.8 Λειτουργία αλγορίθμου

Αρχικά το πρόγραμμα δημιούργει τον πίνακα models στον οποίο αποθηκεύονται ξεχωριστά τα μοντέλα που προσθέτουμε αλλά και τον πίνακα shape που χρησιμοποιείται για την συγκεντρώσει των κορυφών.

Στην συνέχεια μέσα στον βασικό βρόγχο ελέγχεται αν ο χρήστης έχει πατήσει το κουμπί S και ένα το κουμπί έχει πατηθεί δημιουργείτε ένα μοντέλο στο παράθυρο.

Αρχικά πριν την σχεδίαση επιλέγεται τυχαία ένα από τα τέσσερα μοντέλα A, B, C και D με την εντολή rand() % 4.

Αμέσως μετα υπολογίζονται οι τυχαίες συντεταγμενες x και y και καλείτε η συνάρτηση createModel)(η οποία είναι υπεύθυνη για την δημιουργία του μοντέλου

Πριν την αποθήκευση του μοντέλου γίνεται έλεγχος μέσω της συνάρτησης overlap() για το αν υπάρχει επικάλυψη. Σε περιπτώσεις που βρεθεί επικάλυψη ξανά εκτελείτε ο βρόγχος μέχρι να βρεθούν συντεταγμένες στις οποίες δεν επικαλύπτετε το νέο μοντέλο με κάποιο προηγούμενο.

Σε περίπτωση που βρεθεί χώρος χωρίς επικάλυψη το νέο μοντέλο προστίθεται στο πίνακα models με την εντολή models.push_bask(newmodel).

Οι κορυφές των μοντέλων μεταφέρονται στον buffer της OpenGL μέσω της glBufferData() και σχεδιάζονται στην οθόνη με την εντολή glDrawArrays()

Τέλος η διαδικασία επαναλαμβάνετε με κάθε ενώ πάτημα του κουμπιού S μέχρι να εμφανιστούν τρία μοντέλα. Όταν συμπληρωθεί ο μέγιστος αριθμό το πρόγραμμα δεν επιτρέπει να εμφανιστούν αλλά μοντέλα στο παράθυρο. Έτσι εξασφαλίζετε ότι στην οθόνη μπορούν να εμφανίζονται μέχρι τρία μοντέλα.

1.7 Ερώτημα (iv) – διαγραφή παλιότερο μοντέλου με το κουμπί D

1.7.1 Περιγραφή και τρόπος υλοποίησης

Σε αυτό το σημείο της εργασίας μα ζητήθηκε να τροποποιήσουμε το πρόγραμμα ώστε με το πάτημα του κουμπιού D να διαγράφετε το παλιότερο μοντέλο από το παράθυρο.

Για αυτή την υλοποίηση χρησιμοποιήσαμε το πίνακα models στον οποίο τα μοντέλα αποθηκεύονται με την σειρά που σχεδιάστηκαν. Με αυτόν τον τρόπο το πρώτο στοιχείο του πίνακα αντιστοιχεί στο παλιότερο μοντέλο.

Για τον έλεγχο του κουμπιού χρησιμοποιήσαμε τη συνάρτηση glfwGetKey() ενώ για την διαγραφή χρησιμοποιήσαμε την εντολή models.erase(models.begin).

Επιπλέον χρησιμοποιήσαμε τις μεταβλητές currentD και previous ώστε να διαγράφετε ένα μοντέλο ανά πάτημα.

Μετα την διαγραφή γίνετε ανανέωση στον πίνακα shape με τις κορυφές που απομένουν.

1.7.2 Έλεγχος του κουμπιού D

Αρχείο:	Source-1A.cpp
Γραμμές Κώδικα:	273 και 307 – 314

```
273     bool previousD = false;
```

```
307     //elechos gia to pliktro d
308     bool currentD = (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS);
309
310     //diagrafi tou palioterou montelou
311     if (currentD && !previousD && !models.empty()) {
312         models.erase(models.begin());
313     }
314     previousD = currentD;
```

Για να ελέγχουμε πότε ο χρήστης πατάει το κουμπί D χρησιμοποιήσαμε την συνάρτηση glfwGetKey().

Αρχικά δηλώσαμε την μεταβλητή precious η οποία την χρησιμοποιούμαι για την αποθήκευση της προηγούμενης κατάστασης του κουμπιού.

Στην συνέχεια μέσα στο βασικό do/while του προγράμματος προσθέσαμε την εντολή bool currentD = (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS); η οποία την χρησιμοποιούμαι για να αποθηκεύουμε την τρέχουσα κατάσταση του κουμπιού

Μετα η συνθήκη if περιέχει τρεις ελέγχους. Τον currentD ο οποίος έλεγχος είναι για να δούμε αν το κουμπί είναι πατημένο, τον έλεγχο !previousD ο οποίος χρησιμοποιείτε για να δει αν το κουμπί δεν έχει πατηθεί στην προηγούμενή επανάληψη και τον !models.empty() ο οποίος ελέγχει αν ο πίνακας models περιέχει τουλάχιστον ένα μοντέλο.

Τέλος προσθέσαμε την εντολή previous = currentD η οποία ενημερώνει την προηγούμενή κατάσταση του κουμπιού ώστε η διαγραφή να γίνεται μόνο μια φορά ανά πάτημα και όχι συνέχεια.

1.7.3 Διαγραφή του μοντέλου

Αρχείο:	Source-1A.cpp
Γραμμές Κώδικα:	310 – 313

```
310     //diagrafi tou palioterou montelou
311     if (currentD && !previousD && !models.empty()) {
312         models.erase(models.begin());
313     }
```

Για την διαγραφή του παλιότερου μοντέλου χρησιμοποιήσαμε την συνάρτηση erase().

Επίσης μέσα στην συνάρτηση έχουμε model.begin() η οποία επιστρέφει την θέση του πρώτου στοιχείου ενώ η erase() διαγραφεί το στοιχείο που βρίσκετε σε αυτή.

Μετα την διαγραφή τα υπόλοιπα μοντέλα παραμένουν αποθηκευμένα στον πίνακα models. Επιπλέων γίνετε έλεγχος αν υπάρχει τουλάχιστο ένα στοιχείο μέσα στον πίνακα έτσι ώστε να μην γίνει διαγραφεί όταν είναι άδειος.

Τέλος το παράθυρο ενημερώνετε ώστε να εμφανίζονται μόνο τα μοντέλα που έχουν απομείνει και ο χρήστης μπορεί να πατήσει το κουμπί S ξανά για ανά δημιουργήσει ένα νέο μοντέλο.

1.7.4 Λειτουργία αλγορίθμου

Αρχικά γίνεται έλεγχος αν ο χρήστης έχει πατήσει το κουμπί D χρησιμοποιώντας την συνάρτηση glfwGetKey().

Εάν το κουμπί έχει πατηθεί και υπάρχει τουλάχιστον ένα μοντέλο στον πίνακα πραγματοποιείτε η διαγραφή του παλιότερου μοντέλου.

Στην συνέχεια ο πίνακας shape αδειάζει και μαζεύονται ξανά οι κορυφές τον μοντέλων που έχουν απομείνει.

Μετα την διαγραφή ο χρηστή μπορεί να ξανά πατήσει το κουμπί S για να δημιουργήσει νέο μοντέλο εκτός και αν ο συνολικός αριθμός τον μοντέλων είναι μικρότερος από τρία.

Αυτή η διαδικασία μπορεί να γίνετε μέχρι να τερματίσει ο χρήστης το πρόγραμμα ή πατήσει το κουμπί Q.

2. Πληροφορίες σχετικά με την υλοποίηση

2.1 Λειτουργικό σύστημα

Για την ανάπτυξη και τον έλεγχο της εφαρμογής χρησιμοποιήσαμε υπολογιστή με λειτουργικό σύστημα windows.

Η υλοποίηση πραγματοποιήθηκε σε γλώσσα προγραμματισμού C++, και έγινε επίσης η χρήση των βιβλιοθηκών OpenGL, GLFW, GLEW, GLM. Ακόμη χρησιμοποιήσαμε και διαμορφώσαμε κατάλληλα το περιβάλλον του Microsoft Visual Studio.

Το λειτουργικό σύστημα Windows χρησιμοποιήθηκε για να εξασφαλιστεί η σωστή λειτουργεία των κουμπιών S, D και Q.

2.2 Περιβάλλον ανάπτυξης

Για την ανάπτυξη της εφαρμογής χρησιμοποιήσαμε το περιβάλλον του Microsoft Visual Studio και τη γλώσσα C++.

Μέσω του Microsoft Visual Studio πραγματοποιήθηκε το προγραμματιστικό μέρος (coding), η μεταγλώττιση (compile) αλλά και η εκτέλεση (run) του προγράμματος.

Επίσης χρησιμοποιήθηκαν και έγιναν οι κατάλληλες διαμόρφωσης η χρήση των βιβλιοθηκών GLFW, GLEW, GLM η οποίες χρησιμοποιούνται από την OpenGL για την δημιουργία του

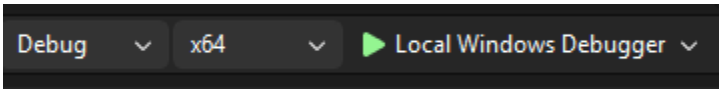
παράθυρου, τη συνεργασία των κουμπιών και του προγράμματος αλλά και για την σχεδίαση μοντέλων.

Επιπλέων χρησιμοποιήσαμε αρχεία shader τα οποία μας ήταν συμπεριλαμβανόμενα στο αρχικό υλικό της άσκησης.

Κατά την διάρκεια ανάπτυξης του προγράμματος εκτελούσαμε ένα ερώτημα ξεχωριστά ώστε να είμαστε σίγουροί για την σωστή λειτουργεία του.

2.3 Ιδιαιτερότητες στον τρόπο εκτελέσεις

Για την εκτέλεση της εφαρμογής ανοίγουμε το αρχείο στο του Microsoft Visual Studio και εκτελούμε το πρόγραμμα χρησιμοποιώντας debugging x64.



Όταν ξεκινήσει το πρόγραμμα εμφανίζεται ένα παράθυρο με διαστάσεις 900x900pixels με μπλε φόντο το οποίο είναι κενό.

Ο χρήστης αλληλοεπιδρά με την εφαρμογή χρησιμοποιώντας τα κουμπιά S, D και Q τα οποία μπορεί να προσθέσει μοντέλα, να αφαιρέσει μοντέλα και να κλίσει την εφαρμογή αντίστοιχα.

Για την σωστή εκτέλεση του προγράμματος ο χρηστή πρέπει να έχει τα αρχεία των shaders στη ίδια διεύθυνση με το αρχείο του κώδικα.

2.4 Άλλες ιδιαιτερότητες

Κατά την ανάπτυξη του προγράμματος αντιμετωπίσαμε δυσκολία σχετικά με την εμφάνιση του τίτλου στο παράθυρο. Συγκεκριμένα ενώ είχαμε ορίσει τίτλο με ελληνικούς χαρακτήρες ο τίτλος του παραθύρου εμφανίζονταν με ερωτηματικά. Όμως το συγκεκριμένο πρόβλημα δεν επηρέαζε την σωστή λειτουργεία του προγράμματος

3. Πληροφορίες σχετικά με την υλοποίηση

Το πρόγραμμα υλοποιήθηκε από ομάδα των δυο ατόμων με στόχο την ολοκλήρωση των τεσσάρων ερωτημάτων.

Η υλοποίηση οργανώθηκε σταδιακά ξεκινώντας από την δημιουργία του παραθύρου και συνεχίζοντας με την σχεδίαση των μοντέλων από το ένα μέλος της ομάδας και τέλειωσε με την

τυχαία τοποθέτηση χωρίς επικάλυψη και τη λειτουργεία διαγραφής από το δεύτερο μέλος της ομάδας.

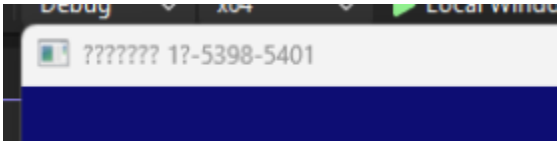
Κατά την διαδικασία υλοποίησης πραγματοποιήθηκαν από κοινού δοκιμές για τον έλεγχο της σωστής λειτουργία του προγράμματος. Τέλος όλες οι λειτουργίες συγκεντρώθηκαν σε ένα αρχείο, στο Source-1A.cpp.

4. Αναφορές

4.1 Επίλυση προβλήματος τίτλου

Κατά την ανάπτυξη του προγράμματος αντιμετωπίσαμε ένα πρόβλημα στο πως εμφανιζόταν ο τίτλος του παράθυρου.

Συγκεκριμένα στην άσκηση είχαμε οδηγίες να εμφανίζετε ο τίτλος με ελληνικούς χαρακτήρες αλλά στο δικό μας παράθυρο εμφανιζόταν ερωτηματικά όπως φαίνεται στην ποιο κάτω φωτογραφία.



Αρχικά επικοινωνήσαμε με την κ. Σταμάτη η οποία μας παρέπεμψε στην ποιο κάτω σελίδα

https://www.glfw.org/docs/3.3/window_guide.html#window_title

όμως δεν καταφέραμε να λύσουμε το πρόβλημα μέσα από τις οδηγίες τις σελίδας. Για αυτό τον λόγο χρησιμοποιήσαμε AI δίνοντας του πληροφορίες για το περιβάλλον ανάπτυξης, τις βιβλιοθήκες που χρησιμοποιούμε και μια φωτογραφία του προβλήματος.

Το ενδεικτικό prompt που χρησιμοποιήσαμε είναι αυτό:

“Echo ena thema sto na tiponete o titlos sta ellinika chrisimopio Microsoft visual studio kai echo ena project stin opnegl I opoioa chrisimopoiei tis vivliothikes GLFW, GLEW kai GLM mou tiponete o titlos me erotimatika opos fenete stin fotgrafia pos sto diorthono”.

Το AI μας πρότεινε να χρησιμοποιήσουμε την επιλογή /utf-8 στις ρύθμισης του project στο Microsoft visual studio

Project -> properties -> configuration properties -> c/c++ -> command line -> additional options.

Το σχετικό σημείο του κώδικα είναι στην γραμμή 206 στο αρχείο Source-1A.cpp (υπάρχουν φωτογραφίες στην επόμενη σελίδα).

```
206     window = glfwCreateWindow(900, 900, "Εργασια 1Α - 5398 - 5401", NULL, NULL);
```

